# MoMIDI: Morse over MIDI

Specification for sending Morse Code over MIDI events.

This document builds on existing MIDI protocols used by several hardware and software manufacturers for triggering keying events between hardware keys, and software keyers, transmitters, practice oscillators, etc. The goal is to use MIDI to trigger key-down and key-up events between morse keys, and software running on a computer (eg: an SDR transmitter, remote station, practice keyer, etc.)

This document represents MoMIDI version **v0.1**

A Quick Reference for the MIDI Protocol: https://www.songstuff.com/recording/article/midi-message-format/

Subscribe to the MoMIDI groups.io email list:

Email Address

View Archives

Subscribe

## Table of Contents


- 1. MoMIDI Specification
  - 1.1. Terms
  - 1.2. Numeric Conventions
  - 1.3. MoMIDI Versions
    - 1.3.1. List of Versions
- 2. MoMIDI Protocol
  - 2.1. Note Values
  - 2.2. MoMIDI Without Timing
  - 2.3. Timing Information
    - 2.3.1. How Time Is Measured Between Events
    - 2.3.2. Representing Time in MIDI
      - 2.3.2.1. No Time Available, or Timer Reset
      - 2.3.2.2. Time Encoding: Base126
      - 2.3.2.3. The largest number that can be represented
  - 2.4. MoMIDI Version Query
    - 2.4.1. What to do with the Version
    - 2.4.2. Ignorant Participants (who don't know MoMIDI versions)
- 3. System Exclusive (SysEx) Manufacturer IDs
  - 3.1. MoMIDI Manufacturer ID table
- 4. Concerns
- 5. Contributors

## Document History

| Date | Version | Author | Changes |
|---|---|---|---|
| 2025-11-24 | n/a | @SmittyHalibut | First draft. |
| 2025-12-03 | n/a | @SmittyHalibut | Updated timing from 1ms to 2ms resolution. Someone should check my math. |
| 2026-01-19 | n/a | @SmittyHalibut | Going back to 1ms resolution. Added pseudocode and tables to Rules. |
| 2026-02-20 | v0.0 | @SmittyHalibut | Changes proposed by @schrockwell: Base126.<br>Added version numbering. |
| 2026-02-25 | v0.0 | @SmittyHalibut | Fixing "off-by-one" confusion on CC=0 Velocity. Not bumping version. |
| 2026-03-20 | v0.0 | @SmittyHalibut | Fixed order of operations in `velocity` calculation. |
| 2026-06-18 | v0.1 | @SmittyHalibut | Issue #1: Timing MSB to Polyphonic Aftertouch. MoMIDI Version to Song Select. Start tracking Manufacturer IDs for SysEx messages. |

# 1. MoMIDI Specification

## 1.1. Terms

- **Key**: In the context of this document, a "key" can be any hardware device meant for sending morse code.
- **Straight Key**: A key who's timing of dits and dahs is done before the conversion to MIDI. Either a literal straight key, a bug, or even a hardware iambic keyer, etc. The important part here is that the straight key has a single electrical contact that controls the transmitter directly.
    - Note: This could also include a hardware iambic keyer. The output of the hardware keyer is a single contact closure, and all the timing is generated on the user side of the MIDI interface.
- **Iambic Key**, or **Paddles**: A key who's timing of dits and dahs is done in software, on the receiving side of the MIDI events.
- **MIDI event**: MIDI, or the Musical Instrument Digital Interface, is a standard for sending very precise timing events over a serial link. It's designed around musical instruments, but we are coopting it for another time and latency sensitive use: sending morse code. The MIDI events used by MoMIDI are:
    - MIDI Note On/Off events: Includes two 7-bit values: Note, and Velocity.
    - MIDI Polyphonic Aftertouch event: Includes two 7-bit values: Note, and Pressure.
    - MIDI Control Change event: Includes two 7-bit values: Channel, and Value.
    - MIDI Song Select event: Includes only one 7-bit value: Song Number.
    - MIDI System Exclusive (SysEx) Start/End events: Includes a 7-bit Manufacturer ID value, and an arbitrary number of 7-bit Data values.

- **Timing Round**: The state machine that tracks timing information. It starts with a time value of zero, and ends after there's been more than `MAX_COUNT` milliseconds since the last tracked event, after which the next event sent will be sent with a value of zero and start a new Timing Round.
- **Sender**: the device sending the MoMIDI packets.
- **Receiver**: the software on the computer receiving and interpreting the MoMIDI packets.

## 1.2. Numeric Conventions

Bare numbers are decimal. Hexadecimal are prefixed with `0x`. eg: `20` is decimal, `0x14` is its hexadecimal equivalent.

## 1.3. MoMIDI Versions

The MoMIDI version indicates what features the receiver can expect from the sender. It is sent as the Value in a Song Select (MIDI Status 0xF3) event, so it's limited to 7 bits.

Those 7 bits are split into Major (3 MSB) and Minor (4 LSB) version numbers:

- Bits 6..4: Major version. Incremented on breaking changes.
- Bits 3..0: Minor version. Incremented on major, but backward compatible changes, and added features.

The version can be written for human consumption using decimal as: `v[major 0..7].[minor 0..15]` For example: `v2.10`.

When writing the version in a technical context, it may also be written as a hexadecimal number: `0x[major 0..7][minor 0..F]` For example: `0x2A` is equivalent to `v2.10`.

### 1.3.1. List of Versions

| Version | Release Date | Description/More Information |
|---|---|---|
| no version | 2025-11-24 | Initial development. Changed a whole lot. |
| 0x00 | 2026-07-01 | MoMIDI Version Query flag. Requests that the receiver respond with its MoMIDI version. |
| 0x01 | 2026-07-01 | Timing MSB to Polyphonic Aftertouch. MoMIDI Version to Song Select. Start tracking Manufacturer IDs for SysEx messages. |

# 2. MoMIDI Protocol

## 2.1. Note Values

MIDI Note On/Off events send a 7-bit Note value, and a 7-bit value called Velocity. It's been pretty well established to use these MIDI Notes to trigger CW events:

- Note On events (MIDI Status 0x9n): key down
- Note Off events (MIDI Status 0x8n): key up
- Note 20: left paddle, (or straight key, depending on the current mode in the Receiver software.)
- Note 21: right paddle
- Note 30: straight key
- Note 31: PTT
- (There are a lot of other MIDI events used for various other radio control features, but we are concentrating on CW here.)

## 2.2. MoMIDI Without Timing

Sending timing information is optional. If no timing information is being tracked, the Sender sends Notes with a Velocity of either 127, or 0. Usually, 127 is sent with Note On events, and 0 with Note Off events. Both of these values mean "no timing information available" and the Receiver software measures the time of events by when the MIDI event was received. The Sender does not need to send, and the Receiver should not expect, any additional MIDI events with timing information. (However, it's not an error if other events are sent for some other reason.)

## 2.3. Timing Information

To send timing information, the Sender measures in firmware the time between key down/up events with millisecond precision. The Sender then sends that information to the software in (one or) two MIDI events:

- The 7 most significant bits (MSB) are sent as the Pressure in a Polyphonic Aftertouch event.
    - This event is optional. If the MSB are 0, this event can be omitted.
- The 7 least significant bits (LSB) as the Velocity of a Note On/Off event.

Remember, Velocities of 0 and 127 mean "no timing information." We can't use those values, leaving 126 usable values in the Velocity field, and the full 128 values in the Pressure field. 126*128 is enough to count over 16 seconds. Beyond that, precise timing isn't critical so we reset the timer and start over.

Calculating the values sent in the Note On/Off Velocity and Aftertouch Pressure are defined below.

Sending a Polyphonic Aftertouch event is optional. When the time is <= 126ms, the Pressure value of the Polyphonic Aftertouch event is 0 and can be omitted. If the Receiver receives a Note On/Off event without a preceding PA event, the Receiver assumes the MSB are 0.

If possible, send the Aftertouch and Note On/Off events in the same USB packet, to minimize latency on the receiver side.

> **Side note**: Above 30wpm (40ms per morse symbol), the MIDI latency between key-down and sound-on in a software based keyer (measured using current hardware and software as: 10ms at best, typically 15ms to 25ms) becomes significant. We assume fast CW operators (above 30wpm) will use hardware based keyers with hardware side-tone.

### 2.3.1. How Time Is Measured Between Events

Timing is tracked between all CW MIDI events, even if they were a different event. For example, when pressing Left, then pressing Right, then releasing Left, and releasing Right, The left-down is sent with 127 signifying the start of timing, the right-down is sent with the time since the left-down, the left-up is sent with the time since the right-down, and the right-up is sent with the time since the left-up.

| Wall clock time | Event | MIDI Sent | Time sent with event |
|---|---|---|---|
| 01:23:45.678 | Left Down | Note On 20 | 127, start of timing. |
| 01:23:45.778 | Right Down | Note On 21 | 100ms |
| 01:23:45.808 | Left Up | Note Off 20 | 30ms |
| 01:23:45.958 | Right Up | Note Off 21 | 150ms |

### 2.3.2. Representing Time in MIDI

Here are the rules used to encode time since last event: 1 .. 16128ms, into MIDI events and values.

#### 2.3.2.1. No Time Available, or Timer Reset

When it's been more than `MAX_COUNT` milliseconds since the last event, the next event is treated as if no timing information is available.

The Sender sends two MIDI events, one of which is optional:

1. **OPTIONAL**: Polyphonic Aftertouch (MIDI Status 0xA0) to the event's Note, with Pressure set to 0.
2. Note On (MIDI Status 0x90) or Off (MIDI Status 0x80) to the event's Note, with Velocity set to 127 (Note On) or 0 (Note Off).

The Receiver considers this event to be "zero milliseconds" as referenced by future events that do send timing information, until another "no timing information available" event is received.

#### 2.3.2.2. Time Encoding: Base126

The number of milliseconds since the last event (here called "Time") is sent in two parts, 7 bits each: the most significant bits in the Pressure of the Polyphonic Aftertouch event, followed by the least significant bits in the Velocity of a Note On/Off event.

Remember, a Note On/Off Velocity of 0 or 127 means "No timing information available" so we can't use those values. That leaves 126 Velocity values, and all 128 Aftertouch Pressures.

Since we can't use Time=0, and to minimize off-by-one confusion while debugging, we consider `Time - 1` when doing the following math. This makes the Note On/Off Velocity equal to the actual time, when the Aftertouch Pressure is calculated to be 0.

The following pseudo-code works for values `1 <= Time <= MAX_VALUE`. Notably, they do NOT work for `Time == 0`. So make sure you handle a zero time explicitly.

- `Polyphonic Aftertouch Pressure = floor((Time-1)/126)`
- `Note On/Off Velocity = (Time-1)%126 + 1`

On the receiving side, you calculate the time as:

- `Time = Velocity + Pressure*126`

A spreadsheet to validate the math can be found here.

Two MIDI events are sent:

1. Polyphonic Aftertouch (MIDI Status 0xA0) to the event's Note, with Pressure set as above. If the Pressure value is 0, this event may be omitted.
2. Event Note On (MIDI Status 0x90) of Off (MIDI Status 0x80) to the event's Note, with the Velocity set as above.

#### 2.3.2.3. The largest number that can be represented

The maximum value that can be represented is: `MAX_COUNT = 128*126 = 16128`. This represents the number of idle milliseconds when the timing gets reset.

## 2.4. MoMIDI Version Query

Either participant, Sender or Receiver, may request that the other indicate what version of the MoMIDI protocol they are using. This part of the protocol is symmetric; either side may initiate the exchange and the other may respond. For writing clarity, we use the terms Requester and Responder here instead of Sender and Receiver.

The Requester sends a Song Select event (MIDI Status 0xF3) with the 7-bit Song Number set to 0. When the Responder receives the Song Select event with a Song Number of 0, it should respond with a Song Select event, with the Song Number set to the MoMIDI version that the Responder supports.

Either side may also offer its version unsolicited by just sending a Song Select event with the Song Number set to its MoMIDI version. There is no acknowledgement required or expected.

### 2.4.1. What to do with the Version

What each party does with this version is up to the developer. It is not required to "negotiate a highest common version" or anything. But if there is backwards-compatibility breaking changes made in the protocol, this version number will let the software select the correct interpretation.

### 2.4.2. Ignorant Participants (who don't know MoMIDI versions)

The Requester should not *require* a response. The Responder(sic) may not respond, and that should not lock-up or confuse the Requester. In this case, the Requester may assume whatever the developer wants. It is suggested that the Requester be conservative with what it expects, and flexible with what it receives.

However, Responders who receive a Song Select event and know about MoMIDI versions, should respond if they are able to.

# 3. System Exclusive (SysEx) Manufacturer IDs

MIDI has the concept of a System Exclusive, or SysEx, message. It represents data that is unique to a particular manufacturer's equipment, and is not meant to be "standardized" across multiple manufacturers. It is often used for things such as firmware updates, audio patch downloads, etc. It can contain large data, and is unique to a specific device or manufacturer.

As such, SysEx messages include a globally recognized "Manufacturer ID" that identifies what type of hardware a given device is. MIDI maintains their own list of manufacturers, which are out-of-scope for MoMIDI. To let MoMIDI manufacturers use SysEx messages to communicate with their hardware, MoMIDI tracks its own list of manufacturers.

MoMIDI manufacturers are responsible for specifying their own protocol for data inside the SysEx stream, since it is by its nature unique to their product. But it is recommended that the SysEx header begin with a Product ID so that a single Manufacturer ID can be used with multiple products. The scope of that Product ID is limited to a given Manufacturer ID.

## 3.1. MoMIDI Manufacturer ID table

At some future date, this may be split out to its own document. For now, we will embed it in the MoMIDI Spec.

| Mfg ID | Manufacturer / Purpose |
|---|---|
| 0x00 | No manufacturer assigned. Use this before your Manufacturer ID is assigned, but don't expect global uniqueness. |
| 0x01 | Lynovations |
| 0x02 | Halibut Electronics |
| 0x03 | Remote Ham Radio |

Contact the MoMIDI groups.io list to request to be assigned a Manufacturer ID.

# 4. Concerns

If you have any concerns with this system, put them here.

- 2025-12-03, @SmittyHalibut: Should this document be kept to morse events only: left/right paddle, and straight key? This is where millisecond level timing is most important. But MIDI is used for SO MUCH MORE, it would be good to get that documented too. 'course, if we do that, then "Morse over MIDI" is an increasingly inappropriate name. I'm open to thoughts on this.

# 5. Contributors

- Lynn Hansen, Lynovations
  - QRZ: KU7Q
- Mark Smith, Halibut Electronics
  - Email: mark-momidi@halibut.com
  - GitHub: @SmittyHalibut
  - QRZ: N6MTS
- Andrew Rodland, NetKeyer
  - Email: andrew@cleverdomain.org
  - GitHub: @arodland
  - QRZ: KC2G
- Rockwell Schrock, Remote Ham Radio
  - Email: rockwell@schrock.me
  - GitHub: @schrockwell
  - QRZ: WW1X
- Dale Farnsworth
  - GitHub: @DaleFarnsworth
  - QRZ: W7DA
- Jonathan Perkins
  - GitHub: @g4ivv
  - QRZ: G4IVV